



MANUAL - Birds Game API



¿Qué es Birds Game?

Birds Game es un juego online donde los usuarios responden preguntas sobre aves de Argentina. El juego identifica fotos de aves y el usuario debe elegir el nombre correcto entre 4 opciones.

Características:

-  Autenticación segura (cuenta personal)
-  Preguntas sobre aves de Argentina
-  Seguimiento de puntuación en tiempo real
-  Ranking global para competir
-  Estadísticas personales detalladas



ÍNDICE

1. [Para Usuarios](#)
2. [Para Desarrolladores](#)
3. [API Reference](#)
4. [Ejemplos Prácticos](#)
5. [Solución de Problemas](#)



PARA USUARIOS

¿Cómo Jugar?

Paso 1: Crear Cuenta

Tienes que registrarte una sola vez:

1. Ve a: `https://tuapp.com/register` (o usa la app)
2. Completa:
 - Nombre: Juan
 - Apellido: Pérez
 - Email: `juan@example.com`
 - Contraseña: `MiContraseña123!` (mínimo 8 caracteres)
3. Haz clic en "Registrarse"
4. ¡Listo! Ya tienes cuenta

¿Qué pasa en backend?

- Se valida que el email no esté registrado
- Se encripta tu contraseña (por seguridad)

- Se guarda tu perfil en la base de datos
 - Se te genera un token de acceso automático
-

Paso 2: Loguearte

Cada vez que inicies sesión:

1. Email: `juan@example.com`
2. Contraseña: `MiContraseña123!`
3. Haz clic en "Ingresar"
4. ¡Ya estás dentro!

¿Qué pasa en backend?

- Se verifica tu email existe
 - Se valida tu contraseña (sin enviarla nunca sin encriptar)
 - Se genera un "token" (como un pase VIP que te identifica)
 - Este token se guarda en tu navegador/app
-

Paso 3: Iniciar a Jugar

Ahora viene la diversión:

1. Haz clic en "Jugar"
2. Se inicia una nueva "sesión de juego"
(es como abrir un nuevo libro de puntuaciones)

¿Qué pasa en backend?

- Se crea una sesión única para ti
 - Se guarda en qué momento empezaste
 - Se reserva un espacio para guardar tus respuestas
 - Estado: "activa" (está en curso)
-

Paso 4: Responder Preguntas

El juego te muestra:

[FOTO DE UN AVE]

¿Cuál es este ave?

- ☐ Hornero
- ☐ Paloma

- ☐ Cotorra
- ☐ Loro

¿Qué pasa en backend?

- El sistema conecta con iNaturalist (base de datos de naturalistas)
- Obtiene una foto de un ave real de Argentina
- Selecciona 1 respuesta correcta
- Genera 3 respuestas falsas al azar
- Mezcla las 4 para que no sepas cuál es la correcta
- Te envía la pregunta

Paso 5: Validación de Respuesta

Cuando respondes:

```
Usuario selecciona: "Hornero"  
Respuesta correcta: "Hornero"
```

✅ ¡CORRECTO! +10 puntos

¿Qué pasa en backend?

- Valida que tengas una sesión activa
- Valida que no hayas respondido ESTA MISMA pregunta ya
- Compara tu respuesta con la correcta
- Si es correcta: +10 puntos
- Si es incorrecta: +0 puntos
- Guarda el resultado con timestamp
- Actualiza tu contador de puntos en vivo

Paso 6: Ver tu Progreso

Durante el juego puedes ver:

```
📊 Estadísticas Actuales  
└ Puntos: 50  
└ Intentos: 6  
└ Aciertos: 5 (83%)
```

¿Qué pasa en backend?

- Suma todos tus puntos en esta sesión
- Cuenta cuántas preguntas respondiste
- Cuenta cuántas acertaste

- Calcula el porcentaje

Paso 7: Terminar Sesión

Cuando acabas de jugar:

1. Haz clic en "Terminar Sesión"
2. Se guarda tu puntuación final
3. Ves tu resumen:
 - └ Puntos totales: 50
 - └ Aciertos: 5 de 6 (83%)

¿Qué pasa en backend?

- Calcula tu puntuación final (suma de todos los puntos)
- Cierra la sesión (estado: "terminada")
- GUARDA permanentemente tu puntuación en el histórico
- Actualiza el ranking global

Paso 8: Ver el Ranking

Puedes compararte con otros jugadores:



RANKING GLOBAL

1. María González - 520 puntos
2. Juan Pérez - 450 puntos ← TÚ ERES AQUÍ
3. Carlos López - 380 puntos

¿Qué pasa en backend?

- Suma TODOS los puntos de TODAS las sesiones terminadas
- Ordena a todos los jugadores de mayor a menor puntuación
- Te muestra tu posición

Paso 9: Ver mis Estadísticas

Puedes ver cómo vas:



MIS ESTADÍSTICAS

Total de intentos: 87
Aciertos: 72
Porcentaje: 82.76%

Puntuación total: 720
Tu posición: #2 en el ranking

¿Qué pasa en backend?

- Suma TODOS tus intentos de TODAS las sesiones
- Suma TODOS tus aciertos
- Calcula el porcentaje general
- Te dice en qué posición estás del ranking

Seguridad

¿Por qué me pide contraseña tan complicada?

- Mínimo 8 caracteres (más caracteres = más seguro)
- Se encripta usando bcrypt (algoritmo militar)
- Ni siquiera los administradores ven tu contraseña real

¿Por qué hay un "token"?

- Es como tu carnet de identificación digital
- Prueba que eres tú sin enviar tu contraseña cada vez
- Expira (por seguridad) y necesitas refrescar

¿Qué datos manejas?

- Nombre, apellido, email
- Intentos y aciertos del juego
- Puntuación
- TODO cifrado y seguro

¿Cómo Ver las Cookies? (Frontend)

El token se guarda en **Cookies HttpOnly** (la forma más segura). Aquí te muestro cómo verificar que está funcionando:

En Google Chrome / Edge / Firefox

1. Abre DevTools

Windows: Presiona F12 o Ctrl+Shift+I
Mac: Cmd+Option+I

2. Ve a la pestaña "Application" (o "Storage")

```
DevTools
├─ Application (Chrome/Edge)
├─ Storage (Firefox)
└─ Haz clic
```

3. En el menú izquierdo, busca "Cookies"

```
Application
├─ Cookies
│   └─ http://localhost:8000 (o tu dominio)
│       └─ Haz clic
```

4. Deberías ver las cookies después de login

Cookies que verás:

Name	Value
access_token	eyJ... (largo)
refresh_token	eyJ... (largo)

Propiedades (en desarrollo):

```
├─ HttpOnly: ☒ (No accesible desde JavaScript)
├─ Secure: ☐ (En desarrollo es HTTP, no HTTPS)
├─ SameSite: Lax
└─ Max-Age: 3600 (1 hora para access_token)
```

Ver en el Network Tab (Más Técnico)

Paso 1: Abre DevTools

```
F12 → Network
```

Paso 2: Haz login

Verás una solicitud POST a /auth/login

Paso 3: Mira la Response

```
POST /auth/login
├─ Response tab:
│   └─ ☒ Verás respuesta JSON
├─ Response Headers:
│   └─ ☒ Verás:
│       Set-Cookie: access_token=eyJ...; HttpOnly; SameSite=Lax; Max-Age=3600
│       Set-Cookie: refresh_token=eyJ...; HttpOnly; SameSite=Lax; Max-Age=604800
```

Paso 4: Verifica que se envía en requests autenticados

```
GET /users/me
├─ Request Headers:
│   └─ Cookie: access_token=eyJ...; refresh_token=eyJ...
│       ☒ Esto se envía automáticamente
├─ Response:
│   └─ ☒ 200 OK - Estás autenticado
```

⚠ Importante: No Puedes Ver el Token Desde JavaScript

```
// ☒ Esto NO funciona (cookies HttpOnly)
console.log(document.cookie)
// Output: (vacío, porque HttpOnly las protege)

// ☒ Esto NO funciona
const token = localStorage.getItem('access_token')
// Output: undefined (no guardamos allá)

// ☒ El navegador lo envía automáticamente
// No necesitas hacer nada especial
```

¿Qué significa HttpOnly?

HttpOnly = "No permitas que JavaScript acceda a esta cookie"

Ventaja:

└─ Si hay malware en la web, NO puede robar el token

Desventaja (para debugging):

└─ No puedes verlo desde console.log()

Solución:

- └ Usa DevTools → Application → Cookies
- Ahí puedes verlo (el navegador lo permite)

Checklist - ¿Funciona bien?

Después de login, verifica:

- ☐ Abre DevTools (F12)
- ☐ Ve a Application → Cookies
- ☐ Ves `access_token` y `refresh_token`
- ☐ Ambas tienen HttpOnly: ☒
- ☐ Vuelve a Network tab
- ☐ Haz una solicitud a `/users/me`
- ☐ En Request Headers ves: `Cookie: access_token=...; refresh_token=...`
- ☐ Response es 200 OK (estás autenticado)

Si todo está ☒ = **Las cookies funcionan correctamente**



PARA DESARROLLADORES

Arquitectura del Sistema

Stack Tecnológico

Frontend

- └ Vue.js / React / Angular
- └ HTTP Requests
- └ JWT en localStorage

Backend (FastAPI)

- └ Python 3.8+
- └ FastAPI (framework web)
- └ SQLAlchemy (ORM)
- └ SQLite (base de datos)

Base de Datos

- └ users (usuarios registrados)
- └ game_sessions (sesiones de juego)
- └ game_results (respuestas individuales)
- └ scores (puntuaciones finales)
- └ scores (puntuaciones históricas)

API Externa

- └ iNaturalist API (datos de aves)
-

Configuración de Tokens

¿Cómo Configurar la Expiración de Tokens?

Los tiempos de expiración se configuran desde el archivo `.env`:

```
# Archivo: .env

# Access Token: tiempo en MINUTOS
# Default: 15 minutos
# Ejemplos:
#   5 = 5 minutos (desarrollo/testing)
#   15 = 15 minutos
#   60 = 1 hora
#   480 = 8 horas
ACCESS_TOKEN_EXPIRE_MINUTES=15

# Refresh Token: tiempo en DÍAS
# Default: 7 días
# Ejemplos:
#   1 = 1 día
#   7 = 7 días (recomendado)
#   30 = 30 días
REFRESH_TOKEN_EXPIRE_DAYS=7
```

Ejemplos Según Uso

Caso de Uso	Access Token	Refresh Token	Razón
Desarrollo/Testing	5 min	1 día	Debuguear sin fricciones
App Web Normal	15 min	7 días	Balance seguridad/UX
App Móvil	15-30 min	30 días	Batería/datos limitados
App Financiera	15 min	1 día	Máxima seguridad

¿Cómo Funciona la Expiración?

Access Token (Corta duración)

```
Usuario hace login → Backend genera access_token (15 min)
↓
Navegador guarda cookie por 15 minutos
↓
Token válido por 15 minutos
↓
Expira → Usuario obtiene 401 Unauthorized
↓
Frontend detecta error 401 → Usa refresh_token
```

↓

Obtiene nuevo access_token automáticamente

Refresh Token (Larga duración)

```
Quando access_token expira:
Frontend hace: POST /auth/refresh
      ↓
Backend valida refresh_token
      ↓
Si es válido:
  └─ Genera nuevo access_token (sin pedir login)
  └─ Usuario continúa usando la app

Si expiró también:
  └─ Usuario debe loguear de nuevo
```

Cambiar Configuración (Paso a Paso)

Paso 1: Edita tu `.env`

```
# Antes
ACCESS_TOKEN_EXPIRE_MINUTES=15
REFRESH_TOKEN_EXPIRE_DAYS=7

# Después (ejemplo: debugging)
ACCESS_TOKEN_EXPIRE_MINUTES=5
REFRESH_TOKEN_EXPIRE_DAYS=1
```

Paso 2: Reinicia el servidor

```
# Detén: Ctrl+C
# Reinicia:
python -m uvicorn app.main:app --reload
```

Paso 3: Verifica que se aplicó

```
python -c "from app.infrastructure.config.cookies import ACCESS_TOKEN_EXPIRE,
REFRESH_TOKEN_EXPIRE; print(f'Access: {ACCESS_TOKEN_EXPIRE}s
({ACCESS_TOKEN_EXPIRE//60}min), Refresh: {REFRESH_TOKEN_EXPIRE}s
({REFRESH_TOKEN_EXPIRE//86400}días)')"
```

Output:

Access: 300s (5min), Refresh: 86400s (1días)

Cómo Verificar en el Navegador

Después de login, abre DevTools y ve:

Application → Cookies → localhost:8000

access_token:

- └ Max-Age: 300 (5 minutos, calculado)
- └ Expira en: [timestamp actual + 300s]

refresh_token:

- └ Max-Age: 86400 (1 día, calculado)
- └ Expira en: [timestamp actual + 86400s]

Flujo Técnico Detallado

1. REGISTRO (POST /auth/register)

CLIENTE ENVÍA:

```
{
  "nombre": "Juan",
  "apellido": "Pérez",
  "email": "juan@example.com",
  "password": "MiContraseña123!"
}
```

SERVIDOR:

1. Valida estructura de datos (Pydantic)
2. Verifica email único
 - └ SELECT FROM users WHERE email = "juan@example.com"
 - └ Si existe: error 400
3. Genera ULID (ID único)
4. Hashea contraseña con bcrypt
5. Inserta en tabla users:
 - └ id: "01ARZ3NDEKTSV4RRFFQ69G5FAV"
 - └ nombre: "Juan"
 - └ email: "juan@example.com"
 - └ password_hash: "\$2b\$12\$..." (no guardamos plaintext)
 - └ role: "jugador"

```
└─ createdAt: 1728226800
└─ is_active: true
6. Genera JWT (access_token + refresh_token)
7. Retorna tokens

CLIENTE RECIBE:
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer"
}

CLIENTE GUARDA:
localStorage.setItem('access_token', access_token)
```

2. LOGIN (POST /auth/login)

```
CLIENTE ENVÍA:
{
  "email": "juan@example.com",
  "password": "MiContraseña123!"
}

SERVIDOR:
1. SELECT FROM users WHERE email = "juan@example.com"
   └─ Si no existe: error 401
2. Compara hash de contraseña
   └─ bcrypt.verify(password, stored_hash)
   └─ Si no coincide: error 401
3. Genera nuevo JWT
4. Retorna tokens

CLIENTE GUARDA:
Authorization: Bearer <access_token>
(En header de todos los requests siguientes)
```

3. INICIAR SESIÓN DE JUEGO (POST /juego/iniciar)

```
CLIENTE ENVÍA:
Authorization: Bearer <access_token>

SERVIDOR:
1. Decodifica JWT → obtiene user_id
2. Verifica que no haya sesión activa ya
3. Genera ULID para session_id
4. Inserta en game_sessions:
   └─ id: "01ARZ3NDEKTSV4RRFFQ69G5FAV"
```

```
└─ user_id: "01ARZ3NDEKTSV4RRFFQ69G5FAV"  
└─ estado: "activa"  
└─ puntos_totales: 0  
└─ total_intentos: 0  
└─ total_aciertos: 0  
└─ createdAt: 1728226900
```

CLIENTE RECIBE:

```
{  
  "session_id": "01ARZ3NDEKTSV4RRFFQ69G5FAV",  
  "mensaje": "Sesión iniciada. Comienza a jugar."  
}
```

CLIENTE GUARDA:

```
sessionStorage.setItem('session_id', session_id)
```

4. OBTENER PREGUNTA (GET /juego/aves)

CLIENTE ENVÍA:

```
GET /api/inaturalist/juego/aves?place_id=10434&locale=es-AR  
Authorization: Bearer <access_token>
```

SERVIDOR:

1. Decodifica JWT
2. Conecta con iNaturalist API
3. Query:
GET https://api.inaturalist.org/v1/observations/species_counts
params:
 - └─ taxon_id: 3 (Aves)
 - └─ place_id: 10434 (Buenos Aires)
 - └─ per_page: 100
4. Procesa respuesta:
 - └─ Extrae aves con foto
 - └─ Extrae nombre común + científico
5. Selecciona 1 al azar:
 - └─ ave_correcta = random.choice(aves)
6. Genera 3 falsas:
 - └─ aves_falsas = random.sample([otras 99 aves], 3)
7. Mezcla las 4:
 - └─ opciones = [correcta] + aves_falsas
 - └─ random.shuffle(opciones)
8. Retorna pregunta

CLIENTE RECIBE:

```
{  
  "ave": {  
    "id": 12345,  
    "nombre": "Hornero",  
    "nombreCientifico": "Furnarius rufus",  
    "foto": "https://...",  
  }  
}
```

```
"opciones": [  
  {"id": 12345, "nombre": "Hornero"},  
  {"id": 67890, "nombre": "Paloma"},  
  {"id": 11111, "nombre": "Cotorra"},  
  {"id": 22222, "nombre": "Loro"}  
]  
}  
}
```

5. RESPONDER PREGUNTA (POST /juego/respuesta)

CLIENTE ENVÍA:

POST /api/inaturalist/juego/respuesta

Authorization: Bearer <access_token>

```
{  
  "ave_id": 12345,  
  "ave_nombre": "Hornero",  
  "respuesta_usuario": "Hornero"  
}
```

SERVIDOR:

1. Decodifica JWT → user_id
2. Obtiene sesión activa:
 - └ SELECT FROM game_sessions WHERE user_id = ? AND estado = "activa"
 - └ Si no existe: error 404
3. Valida respuesta única:
 - └ SELECT FROM game_results WHERE session_id = ? AND ave_id = ?
 - └ Si existe: error 400 "Ya respondiste esta pregunta"
4. Compara respuestas:
 - └ "Hornero".lower() == "Hornero".lower() → True
5. Asigna puntos:
 - └ es_correcta = True
 - └ puntos = 10
6. Inserta en game_results:
 - └ id: "01ARZ3NDEKTSV4RRFFQ69G5FAV"
 - └ user_id: "01ARZ3NDEKTSV4RRFFQ69G5FAV"
 - └ session_id: "01ARZ3NDEKTSV4RRFFQ69G5FAV" ← IMPORTANTE
 - └ ave_id: 12345
 - └ ave_nombre: "Hornero"
 - └ respuesta_usuario: "Hornero"
 - └ es_correcta: true
 - └ puntos_obtenidos: 10
 - └ createdAt: 1728226950

CLIENTE RECIBE:

```
{  
  "es_correcta": true,  
  "puntos_obtenidos": 10,
```

```
"mensaje": "¡Correcto! ✓"
}
```

6. TERMINAR SESIÓN (POST /juego/terminar)

CLIENTE ENVÍA:

```
POST /api/inaturalist/juego/terminar
Authorization: Bearer <access_token>
```

SERVIDOR:

1. Decodifica JWT → user_id
2. Obtiene sesión activa
3. Calcula totales:
 - └ SELECT SUM(puntos_obtenidos) FROM game_results
WHERE session_id = ?
 - └ SELECT COUNT(*) FROM game_results
WHERE session_id = ? AND es_correcta = true
 - └ SELECT COUNT(*) FROM game_results
WHERE session_id = ?
4. Actualiza game_sessions:
 - └ puntos_totales = 50
 - └ total_aciertos = 5
 - └ total_intentos = 6
 - └ estado = "terminada"
 - └ terminadaAt = 1728227100
5. Inserta en scores (PERMANENTE):
 - └ id: "01ARZ3NDEKTSV4RRFFQ69G5FAV"
 - └ user_id: "01ARZ3NDEKTSV4RRFFQ69G5FAV"
 - └ puntos: 50 ← GUARDADO PARA SIEMPRE
 - └ juego: "birds_game"
 - └ fecha: 1728227100
 - └ createdAt: 1728227100

CLIENTE RECIBE:

```
{
  "session_id": "01ARZ3NDEKTSV4RRFFQ69G5FAV",
  "puntos_totales": 50,
  "total_intentos": 6,
  "total_aciertos": 5,
  "porcentaje_acierto": 83.33,
  "mensaje": "Sesión finalizada. Puntuación guardada."
}
```

7. VER RANKING (GET /juego/ranking)

CLIENTE ENVÍA:

```
GET /api/inaturalist/juego/ranking?limit=10
```

```
Authorization: Bearer <access_token>
```

SERVIDOR:

1. Decodifica JWT
2. Query compleja:

```
SELECT
  users.id,
  users.nombre,
  users.apellido,
  SUM(scores.puntos) as puntos_totales,
  COUNT(CASE WHEN game_results.es_correcta = true) as aciertos,
  COUNT(game_results.id) - COUNT(...) as fallos
FROM users
LEFT JOIN scores ON users.id = scores.user_id
LEFT JOIN game_results ON users.id = game_results.user_id
GROUP BY users.id
ORDER BY puntos_totales DESC
LIMIT 10
```
3. Ordena por puntos DESC (mayor primero)
4. Retorna top 10

CLIENTE RECIBE:

```
{
  "total_jugadores": 42,
  "ranking": [
    {
      "user_id": "...",
      "nombre": "María",
      "apellido": "González",
      "puntos_totales": 520,
      "aciertos": 52,
      "fallos": 18
    },
    ...
  ]
}
```

Modelos de Base de Datos

Tabla: users

```
users
├─ id (String, PK) - ULID único
├─ nombre (String)
├─ apellido (String)
├─ email (String, UNIQUE, INDEX)
├─ password_hash (String) - bcrypt hash
├─ role (String) - "jugador", "mod", "admin"
├─ is_active (Boolean)
```

```
| createdAt (Int) - Unix timestamp
| updatedAt (Int)
| lastLogin (Int, nullable)
```

Tabla: game_sessions (NUEVA)

```
game_sessions
| id (String, PK) - ULID único
| user_id (String, FK) → users.id
| puntos_totales (Int) - Calculado al terminar
| total_intentos (Int) - Calculado al terminar
| total_aciertos (Int) - Calculado al terminar
| estado (String) - "activa" | "terminada"
| createdAt (Int) - Unix timestamp
| terminadaAt (Int, nullable)
```

Tabla: game_results (MODIFICADA)

```
game_results
| id (String, PK) - ULID único
| user_id (String, FK) → users.id
| session_id (String, FK) → game_sessions.id [NUEVO]
| ave_id (Int)
| ave_nombre (String)
| respuesta_usuario (String)
| es_correcta (Boolean)
| puntos_obtenidos (Int)
| createdAt (Int) - Unix timestamp
```

Tabla: scores (YA EXISTÍA, AHORA SE USA)

```
scores
| id (String, PK) - ULID único
| user_id (String, FK) → users.id
| puntos (Int) - Puntuación total de la sesión
| juego (String) - "birds_game"
| fecha (Int) - Cuándo se terminó la sesión
| createdAt (Int) - Unix timestamp
```

Validaciones Clave

1. Validación de Sesión Activa

```
# Antes de responder
sesion = db.query(GameSessionModel).filter(
    GameSessionModel.user_id == user_id,
    GameSessionModel.estado == "activa"
).first()

if not sesion:
    raise HTTPException(404, "No hay sesión activa")
```

2. Validación de Respuesta Única

```
# Evitar responder 2 veces la misma pregunta
existe = db.query(GameResultModel).filter(
    GameResultModel.session_id == session_id,
    GameResultModel.ave_id == ave_id
).first()

if existe:
    raise HTTPException(400, "Ya respondiste esta pregunta")
```

3. Autenticación JWT

```
# Verificar que el token es válido
payload = decode_token(token)

if not payload or "sub" not in payload:
    raise HTTPException(401, "Token inválido")

user_id = payload["sub"]
```



API REFERENCE

Endpoints Resumen

Auth (Sin protección)

```
POST /auth/register
POST /auth/login
POST /auth/refresh
```

Users (Con protección 🔒)

```
🔒 GET /users/me
🔒 PATCH /users/me/password
🔒 GET /users
🔒 GET /users/{user_id}
```

Game (Con protección 🔒)

```
🔒 POST /api/inaturalist/juego/iniciar
🔒 GET /api/inaturalist/juego/aves
🔒 POST /api/inaturalist/juego/respuesta
🔒 POST /api/inaturalist/juego/terminar
🔒 GET /api/inaturalist/juego/sesion/estadisticas
🔒 GET /api/inaturalist/juego/ranking
```

Metrics (Con protección 🔒)

```
🔒 GET /api/inaturalist/metricas/usuario
🔒 GET /api/inaturalist/metricas/global
```



EJEMPLOS PRÁCTICOS

Ejemplo 1: Flujo Completo Desde Cero

```
# 1. REGISTRARSE
curl -X POST http://localhost:8000/auth/register \
  -H "Content-Type: application/json" \
  -d '{
    "nombre": "Juan",
    "apellido": "Pérez",
    "email": "juan@example.com",
    "password": "MiContraseña123!"
  }'

# Response:
# {
#   "access_token": "eyJ...",
#   "refresh_token": "eyJ...",
#   "token_type": "bearer"
# }

TOKEN="eyJ..." # Guardar el token

# 2. INICIAR SESIÓN DE JUEGO
```

```
curl -X POST http://localhost:8000/api/inaturalist/juego/iniciar \
-H "Authorization: Bearer $TOKEN"

# Response:
# {
#   "session_id": "01ARZ...",
#   "mensaje": "Sesión iniciada. Comienza a jugar."
# }

SESSION_ID="01ARZ..." # Guardar session_id

# 3. OBTENER PREGUNTA
curl -X GET "http://localhost:8000/api/inaturalist/juego/aves?place_id=10434" \
-H "Authorization: Bearer $TOKEN"

# Response:
# {
#   "ave": {
#     "id": 12345,
#     "nombre": "Hornero",
#     "nombreCientifico": "Furnarius rufus",
#     "foto": "https://...",
#     "opciones": [...]
#   }
# }

# 4. RESPONDER (Usuario selecciona "Hornero")
curl -X POST http://localhost:8000/api/inaturalist/juego/respuesta \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "ave_id": 12345,
  "ave_nombre": "Hornero",
  "respuesta_usuario": "Hornero"
}'

# Response:
# {
#   "es_correcta": true,
#   "puntos_obtenidos": 10,
#   "mensaje": "¡Correcto! ✓"
# }

# 5. [REPETIR PASOS 3-4 VARIAS VECES]

# 6. TERMINAR SESIÓN
curl -X POST http://localhost:8000/api/inaturalist/juego/terminar \
-H "Authorization: Bearer $TOKEN"

# Response:
# {
#   "session_id": "01ARZ...",
#   "puntos_totales": 50,
#   "total_intentos": 6,
```

```
# "total_aciertos": 5,  
# "porcentaje_acierto": 83.33,  
# "mensaje": "Sesión finalizada. Puntuación guardada."  
# }  
  
# 7. VER RANKING  
curl -X GET "http://localhost:8000/api/inaturalist/juego/ranking?limit=10" \  
-H "Authorization: Bearer $TOKEN"  
  
# Response:  
# {  
#   "total_jugadores": 42,  
#   "ranking": [  
#     {  
#       "user_id": "...",  
#       "nombre": "María",  
#       "apellido": "González",  
#       "puntos_totales": 520,  
#       ...  
#     }  
#   ]  
# }
```

Ejemplo 2: Error - Intentar Responder 2 Veces la Misma Pregunta

```
# Primera respuesta (OK)  
curl -X POST http://localhost:8000/api/inaturalist/juego/respuesta \  
-H "Authorization: Bearer $TOKEN" \  
-d '{"ave_id": 12345, "ave_nombre": "Hornero", "respuesta_usuario":  
"Hornero"}'  
  
# Response:   
# {"es_correcta": true, "puntos_obtenidos": 10, "mensaje": "¡Correcto! ✓"}  
  
# Segunda respuesta MISMA PREGUNTA (ERROR)  
curl -X POST http://localhost:8000/api/inaturalist/juego/respuesta \  
-H "Authorization: Bearer $TOKEN" \  
-d '{"ave_id": 12345, "ave_nombre": "Hornero", "respuesta_usuario":  
"Hornero"}'  
  
# Response:  ERROR 400  
# {  
#   "detail": "Ya has respondido esta pregunta en esta sesión"  
# }
```



SOLUCIÓN DE PROBLEMAS

Error: "401 Unauthorized"

Causa: Token no válido o expirado

Solución:

```
# 1. Verifica que includes el header Authorization
-H "Authorization: Bearer <token>"

# 2. Si el token expiró, refrésca lo:
curl -X POST http://localhost:8000/auth/refresh \
  -H "Authorization: Bearer $REFRESH_TOKEN"

# 3. Si seguís teniendo problemas, logueate de nuevo
curl -X POST http://localhost:8000/auth/login \
  -d '{"email": "...", "password": "..."}'
```

Error: "No hay una sesión activa"

Causa: Olvidaste hacer POST /juego/iniciar

Solución:

```
# Antes de responder preguntas, SIEMPRE iniciar sesión:
curl -X POST http://localhost:8000/api/inaturalist/juego/iniciar \
  -H "Authorization: Bearer $TOKEN"
```

Error: "Ya has respondido esta pregunta en esta sesión"

Causa: Respondiste la MISMA pregunta 2 veces

Solución:

- Obtén una nueva pregunta: GET /juego/aves
- Responde una diferente

Error: "No hay una sesión activa. Inicia una con POST /juego/iniciar"

Causa: La sesión fue cerrada

Solución:

```
# Termina la sesión vieja:
curl -X POST http://localhost:8000/api/inaturalist/juego/terminar \
  -H "Authorization: Bearer $TOKEN"
```

```
# Inicia una nueva:
curl -X POST http://localhost:8000/api/inaturalist/juego/iniciar \
  -H "Authorization: Bearer $TOKEN"
```

Error: "El email ya está registrado"

Causa: Ese email ya tiene cuenta

Solución:

- Usa otro email para registrarte
- O logueate con ese email: `POST /auth/login`

Error: "La contraseña es muy débil"

Causa: Contraseña menos de 8 caracteres

Solución:

- Contraseña mínimo 8 caracteres
- Ejemplo: "MiContraseña123!" 

¿Por qué mi puntuación no aparece en el ranking?

Motivo: Solo apareces si TERMINASTE la sesión

Solución:

```
# ANTES de terminar:
- Responde preguntas
- Tu puntuación está en: GET /sesion/estadisticas (temporal)

# DESPUÉS de terminar:
- Tu puntuación se guarda permanentemente
- Apareces en: GET /ranking
```

¿Cómo veo mis datos de usuario?

```
curl -X GET http://localhost:8000/users/me \
  -H "Authorization: Bearer $TOKEN"

# Response:
# {
#   "id": "...",
```

```
# "nombre": "Juan",
# "apellido": "Pérez",
# "email": "juan@example.com",
# "role": "jugador",
# "is_active": true,
# "createdAt": 1728226800
# }
```

¿Cómo veo mis estadísticas generales?

```
curl -X GET http://localhost:8000/api/inaturalist/metricas/usuario \
-H "Authorization: Bearer $TOKEN"

# Response:
# {
#   "total_intentos": 87,
#   "total_aciertos": 72,
#   "porcentaje_acierto": 82.76,
#   "puntuacion_total": 720,
#   "posicion_ranking": 2
# }
```



CHECKLIST DE CONFIGURACIÓN

Para Usuarios

- ☐ He creado una cuenta
- ☐ Puedo loguearme
- ☐ Puedo ver mis datos
- ☐ Puedo iniciar una sesión de juego
- ☐ Puedo responder preguntas
- ☐ Puedo ver el ranking
- ☐ Puedo cambiar mi contraseña

Para Desarrolladores

- ☐ Backend levanta sin errores
 - ☐ Base de datos está creada
 - ☐ Puedo conectarme con curl
 - ☐ Puedo obtener un token JWT
 - ☐ Puedo responder una pregunta
 - ☐ Puedo ver el ranking
 - ☐ Todos los endpoints responden 200 OK
-



Ahora entiendes completamente cómo funciona **Birds Game**.

¿**Dudas?** Revisa la sección de "Solución de Problemas" o contacta al equipo de desarrollo.

¿**Quieres contribuir?** El código está en GitHub: [link al repo]